TOPIC 01

Data Preprocessing with scikit-learn

Introduction to scikit-learn, then the concepts in the class script, explained with your Data.csv

Part 1. Introduction to scikit-learn

1.1 Definition

> **DEFINITION** **scikit-learn**
>
> A free, open-source **Python library for machine learning** ('Machine Learning in Python'). It provides simple and efficient ready-made tools for **predictive data analysis**: preprocessing, classification, regression, clustering, dimensionality reduction and model evaluation. It is imported in code as `sklearn`.

1.2 Why it is needed

- **Ready-made algorithms.** Writing linear regression, decision trees or scaling from scratch takes days and is error-prone; scikit-learn gives tested versions in a line or two.
- **One consistent interface.** Every tool is used through the same few methods (`fit`, `transform`, `predict`), so switching from one algorithm to another changes one line, not the whole program.
- **Fits the Python data stack.** It takes NumPy arrays and Pandas data directly and returns NumPy arrays.
- **Covers the whole workflow.** Cleaning, splitting, training, evaluating and tuning are all in one library.
- **Free and well documented.** BSD licence (usable commercially), with documentation and examples.

1.3 Background

Fact	Detail
Origin	Started in 2007 as a Google Summer of Code project by David Cournapeau
First public release	1 February 2010, by a team at INRIA (Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel)
Built on	NumPy (arrays), SciPy (scientific computing), matplotlib (plots)
Licence	BSD (free, also for commercial use)
Reference paper	Pedregosa et al., 'Scikit-learn: Machine Learning in Python', JMLR, 2011

1.4 What it offers

Area	What it does	Module (sklearn.xxx)	Example tools
Preprocessing	Prepare and transform data	`impute`, `preprocessing`	SimpleImputer, LabelEncoder, StandardScaler
Model selection	Split data, validate, tune	`model_selection`	train_test_split, cross-validation, grid search
Regression	Predict a number	`linear_model`, `ensemble`	LinearRegression, Ridge, Lasso, random forest
Classification	Predict a category	`linear_model`, `tree`, `neighbors`, `svm`	LogisticRegression, decision tree, KNN, SVM
Clustering	Group similar rows (no Y)	`cluster`	KMeans, hierarchical clustering
Dimensionality reduction	Fewer columns	`decomposition`	PCA
Metrics	Score a model	`metrics`	R-squared, accuracy, confusion matrix

1.5 The common interface (how every tool is used)

scikit-learn objects are called **estimators**. Each one is created, then **fitted** to data, then **applied**. Three kinds of object share this pattern:

Kind	Meaning	Key methods	Examples in this script
Transformer	Changes the data	`fit`, `transform`, `fit_transform`	SimpleImputer, LabelEncoder, StandardScaler
Predictor (model)	Learns from X and Y and predicts	`fit`, `predict`, `score`	LinearRegression (used in later topics)
Splitter / utility	Helper functions	called directly	train_test_split

```
obj = SomeTool()           # 1. create the object (choose its settings)
obj.fit(X_train)           # 2. learn from the training data
obj.transform(X_new)       # 3a. transformers: apply what was learned
obj.predict(X_new)         # 3b. models: predict
```

`fit` **learns** (a mean, a mapping, or model weights); `transform` or `predict` **uses** what was learned; `fit_transform` does fit and transform in one call.

1.6 Installing and importing

```
pip install scikit-learn          # install once (the package name)
from sklearn.impute import SimpleImputer   # import name is sklearn
```

The install name is `scikit-learn`, but the import name is `sklearn`. Classes are imported from the module they belong to, which is why your script has a separate import line for each tool.

1.7 Where your script uses it

Step	Module	Class or function	Kind
Fill blanks	`sklearn.impute`	`SimpleImputer`	Transformer
Encode labels	`sklearn.preprocessing`	`LabelEncoder`	Transformer
Split rows	`sklearn.model_selection`	`train_test_split`	Utility function
Scale columns	`sklearn.preprocessing`	`StandardScaler`	Transformer

1.8 Limits to know

- It is for **classical machine learning**. Deep neural networks are normally built with other libraries such as TensorFlow or PyTorch.
- It does not load files or clean tables itself; **Pandas** does that, and scikit-learn takes over from there.

Part 2. Data preprocessing with Data.csv

The data

`Data.csv` has 10 rows and 4 columns. The first three are the **independent variables (X)**; the last is the **dependent variable (Y)** to predict. Two values are missing (marked in bold).

#	State	Age	Salary	Purchased
0	Mumbai	44	72,000	No
1	Bangalore	27	48,000	Yes
2	Hyderabad	30	54,000	No
3	Bangalore	38	61,000	No
4	Hyderabad	40	**missing**	Yes
5	Mumbai	35	58,000	Yes
6	Bangalore	**missing**	52,000	No
7	Mumbai	48	79,000	Yes
8	Hyderabad	50	83,000	No
9	Mumbai	37	67,000	Yes

> **DEFINITION** **Data preprocessing**
> The set of steps that **transform raw data into a clean, numeric, consistently scaled form** that a machine learning model can learn from. Several steps applied in a fixed order are called a **preprocessing pipeline**.

Why it is needed: a model cannot use this table as it is. It has blanks, text (State, Purchased) and columns on very different scales (Age in tens, Salary in tens of thousands). The script fixes this in five steps.

Step	Concept	Tool
1	Load the file and separate X and Y	`pd.read_csv`, `iloc`
2	Fill missing values	`SimpleImputer`
3	Convert text to numbers	`LabelEncoder`
4	Split into training and test sets	`train_test_split`
5	Bring columns to the same scale	`StandardScaler`

2.1 Loading the data and separating X and Y

```
dataset = pd.read_csv("Data.csv")
x = dataset.iloc[:, :-1].values   # all rows, all columns except the last
y = dataset.iloc[:, 3].values     # all rows, column 3
```

> **DEFINITION** **DataFrame, iloc, X and Y**
> A **DataFrame** is Pandas' table of rows and columns. `iloc[rows, columns]` selects part of it by **position**. **X** (independent variables) are the input columns; **Y** (dependent variable) is the column to be predicted.

Why it is needed: a supervised model learns the link between inputs and the known answer, so it must receive X and Y as two separate objects.

How it works: `read_csv` loads the file into a DataFrame. `iloc[:, :-1]` means all rows and every column except the last; `iloc[:, 3]` means all rows and column number 3 (counting from 0). `.values` converts the selection into a NumPy array, the format scikit-learn expects.

On this data: x = State, Age, Salary (10 × 3). y = Purchased (10 values).

2.2 Missing values and SimpleImputer

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer()
imputer = imputer.fit(x[:, 1:3])
x[:, 1:3] = imputer.transform(x[:, 1:3])
```

> **DEFINITION** **Missing value**
> A cell with no recorded value, shown as blank or **NaN** (Not a Number). It appears when data was not collected, was lost, or does not apply.

> **DEFINITION** **SimpleImputer**
> A scikit-learn class (`sklearn.impute`) that **replaces missing values with a simple statistic of the same column**: the mean (default), median, most frequent value, or a constant. Replacing missing values with estimates is called **imputation**.

Why it is needed:

- Most models cannot accept blanks; they stop with an error.
- Deleting the row loses its other, valid values. With only 10 rows, removing 2 would discard 20% of the data.
- A typical value keeps the row while adding little distortion to the column.

- The imputer remembers the statistic, so **new data with blanks can be filled in the same way later**.

How it works: `fit` **learns** the statistic (the mean) of each column from its known values; `transform` **writes** it into the blanks. Only columns 1 and 2 (Age, Salary) are passed in, because a mean exists only for numbers.

On this data:

Column	Known values	Mean learned	Blank filled
Age	9 (row 6 is blank)	349 / 9 = **38.78**	Row 6: 38.78
Salary	9 (row 4 is blank)	574,000 / 9 = **63,777.78**	Row 4: 63,777.78

Limitation: extreme values pull the mean; the median is safer with outliers.

2.3 Categorical data and LabelEncoder

```
from sklearn.preprocessing import LabelEncoder
labelencoder_x = LabelEncoder()
labelencoder_y = LabelEncoder()
x[:, 0] = labelencoder_x.fit_transform(x[:, 0])
y = labelencoder_y.fit_transform(y)
```

> **DEFINITION** **Categorical variable**
> A variable whose values are **labels or groups** rather than quantities, such as State (Mumbai, Bangalore, Hyderabad) or Purchased (Yes, No).

> **DEFINITION** **LabelEncoder**
> A scikit-learn class (`sklearn.preprocessing`) that **converts each distinct label of a column into an integer**, numbering the labels in alphabetical order. This conversion is called **encoding**.

Why it is needed: models are built on arithmetic (sums, distances, products). They cannot calculate with the word 'Mumbai', so every column, including Y, must be numeric before training.

How it works: `fit` finds the distinct labels and sorts them; `transform` replaces each label by its position in that list. `fit_transform` does both in one call. Two encoders are used because State (X) and Purchased (Y) have different sets of labels, and `inverse_transform` can turn predicted numbers back into labels.

On this data:

Column	Mapping
State (X)	Bangalore = 0, Hyderabad = 1, Mumbai = 2
Purchased (Y)	No = 0, Yes = 1

Limitation: the numbers imply an order (Mumbai 'greater than' Bangalore) that cities do not have. This is harmless for the two-label Y but can mislead a model on X; the usual remedy for unordered X columns is **one-hot encoding** (one yes/no column per category).

2.4 Train and test split

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x, y, train_size=0.8, random_state=0)
```

> **DEFINITION** **Train and test split**
> Randomly dividing the dataset into a **training set**, which the model learns from, and a **test set**, which is kept aside and used only to judge the finished model on unseen data. `train_test_split` is the scikit-learn function that does this.

Why it is needed: the goal of ML is to predict **new** cases. A model scored on the rows it learned from can simply memorise them and look perfect (overfitting). Test rows act as stand-ins for the future and give an honest estimate of real performance.

How it works: the rows are shuffled and cut into two groups. `train_size=0.8` puts 80% in training. `random_state=0` fixes the shuffle, so the **same rows land in the same group every run**, which makes results repeatable. X and Y are split together so each row keeps its answer.

On this data: 8 rows train (rows 0, 1, 3, 4, 5, 6, 7, 9) and 2 rows test (rows 2 and 8). Shapes: **x_train (8, 3)**, **x_test (2, 3)**.

2.5 Feature scaling and StandardScaler

$$z = \frac{x - \mu}{\sigma}$$

```
from sklearn.preprocessing import StandardScaler
sc_x = StandardScaler()
x_train = sc_x.fit_transform(x_train)
x_test  = sc_x.transform(x_test)
```

> **DEFINITION** **Feature scaling and StandardScaler**
> **Feature scaling** puts columns with different units onto a comparable scale. `StandardScaler` (`sklearn.preprocessing`) does this by **standardisation**: each value becomes the number of standard deviations it lies from its column mean (the z-score above), so every column ends with **mean 0 and standard deviation 1**.

Why it is needed: Salary (about 60,000) is over a thousand times larger than Age (about 40). Models that use distances or gradient-based training would let Salary dominate only because of its units, not because it matters more. Scaling gives each column a fair say and usually makes training faster and more stable.

How it works: `fit_transform` on the **training** data learns each column's mean (μ) and standard deviation (σ) and applies the formula. `transform` on the **test** data reuses those same μ and σ. The test data must never be used to fit: that would let information about the test rows leak into training and spoil the honest evaluation from Section 2.4.

On this data (learned from the 8 training rows):

Column	Mean (μ)	Std (σ)
State code	1.125	0.927
Age	38.472	5.808
Salary	62,597.22	9,534.85

Check by hand: test row 2 has Age 30, so z = (30 − 38.472) / 5.808 = **−1.459**. Its Salary 54,000 gives (54,000 − 62,597.22) / 9,534.85 = **−0.902**, and its State code 1 (Hyderabad) gives (1 − 1.125) / 0.927 = **−0.135**. The script's test output for that row is exactly (−0.135, −1.459, −0.902).

2.6 Summary

Step	Before	After
Impute	Blank Age (row 6) and Salary (row 4)	38.78 and 63,777.78
Encode	State and Purchased as text	State 0 to 2; Purchased 0 or 1
Split	10 rows	8 train, 2 test
Scale	Age tens, Salary tens of thousands	Mean 0, std 1 (training data)
Rule	`fit` learns from training data only	`transform` reuses the same numbers on test data